

Learning

Deep Learning · AI & Big Data Engineering · UIR · Prof. H. Hafidi

Graded lab — total: 20 points · Estimated time: ~3 hours

In Lecture 3 you pushed an MLP on FashionMNIST to roughly **88–89% test accuracy** and watched it plateau. The MLP simply does not exploit the *spatial structure* of an image — every pixel is just one input among 784. Today you will break that ceiling in two different ways:

1. **Exercise 1 — Build a small CNN from scratch** that learns local visual features and beats the MLP.
2. **Exercise 2 — Transfer learning with ResNet50** pretrained on ImageNet, adapted to FashionMNIST.

By the end you should have a clear, evidence-based answer to: *“When is it worth training a CNN from scratch, and when should I reach for a pretrained model?”*

 0 · Setup

 Note

If you are running on **Google Colab**, switch to a GPU runtime: [Runtime → Change runtime type → T4 GPU](#).

```
# Colab/local-friendly: install only if needed
import importlib, subprocess, sys
for pkg in ['torch', 'torchvision']:
    if importlib.util.find_spec(pkg) is None:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', pkg])
```

```
import os, time, random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
from cycler import cycler

# ----- UIR matplotlib config -----
UIR_COLORS = {
    'navy': '#0F3B63',
    'lime': '#CDD516',
    'gray': '#6B7D8D',
    'teal': '#2E8B87',
    'coral': '#E07A5F',
}

plt.rcParams.update({
    'figure.figsize': (10, 6),
    'figure.facecolor': 'white',
    'axes.facecolor': 'white',
    'axes.edgecolor': '#1E2A38',
    'axes.labelcolor': '#1E2A38',
    'axes.titlecolor': '#0F3B63',
    'axes.titleweight': 'bold',
    'axes.titlesize': 14,
    'axes.labelsizsize': 12,
    'axes.grid': True,
```

```
'grid.color':      '#E8EBF0',
'grid.linestyle':  '--',
'grid.alpha':      0.6,
'xtick.color':     '#1E2A38',
'ytick.color':     '#1E2A38',
'font.family':     'sans-serif',
'font.size':       11,
'axes.prop_cycle':  cycler(color=list(UIR_COLORS.values())),
})

# ----- Reproducibility -----
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'PyTorch {torch.__version__} | device: {DEVICE}')
```

```
PyTorch 2.10.0+cu128 | device: cuda
```

 1 · Data — FashionMNIST

FashionMNIST — 10 clothing categories, 28×28 grayscale, 60k train / 10k test. We will use it for **both** exercises so the comparison between models is clean.

```
help(torch.conv2d)

Help on built-in function conv2d in module torch:

conv2d(...)
    conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1) -> Tensor

    Applies a 2D convolution over an input image composed of several input
    planes.

    This operator supports :ref:`TensorFloat32<tf32_on_ampere>`.

    See :class:`~torch.nn.Conv2d` for details and output shape.

    Note:
        In some circumstances when given tensors on a CUDA device and using CuDNN, t

    Note:
        This operator supports complex data types i.e. ``complex32, complex64, compl

    Args:
        input: input tensor of shape :math:`(\text{minibatch} , \text{in\_channels})`
        weight: filters of shape :math:`(\text{out\_channels} , \frac{\text{in\_chan}}{\text{stride}})`
        bias: optional bias tensor of shape :math:`(\text{out\_channels})`. Default:
        stride: the stride of the convolving kernel. Can be a single number or a
            tuple `(sH, sW)`. Default: 1
        padding: implicit paddings on both sides of the input. Can be a string {'val
            single number or a tuple `(padH, padW)`. Default: 0
            ``padding='valid'`` is the same as no padding. ``padding='same'`` pads
            the input so the output has the same shape as the input. However, this mod
            doesn't support any stride values other than 1.

        .. warning::
            For ``padding='same'``, if the ``weight`` is even-length and
            ``dilation`` is odd in any dimension, a full :func:`pad` operation
            may be needed internally. Lowering performance.

        dilation: the spacing between kernel elements. Can be a single number or
            a tuple `(dH, dW)`. Default: 1
        groups: split input into groups, both :math:`\text{in\_channels}` and :math:
            should be divisible by the number of groups. Default: 1

    Examples::

    >>> # With square kernels and equal stride
```

```
>>> filters = torch.randn(8, 4, 3, 3)
>>> inputs = torch.randn(1, 4, 5, 5)
>>> F.conv2d(inputs, filters, padding=1)
```

```
# Native FashionMNIST: 28x28, 1 channel, in [0,1]. We normalize to roughly zero-mean
FM_MEAN, FM_STD = 0.2860, 0.3530
```

```
tfm_native = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((FM_MEAN,), (FM_STD,)),
])
```

```
DATA_ROOT = './data'
```

```
train_native = datasets.FashionMNIST(DATA_ROOT, train=True, download=True, transform=tfm_native)
test_native = datasets.FashionMNIST(DATA_ROOT, train=False, download=True, transform=tfm_native)
```

```
CLASSES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
            'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
print(f'Train: {len(train_native):,} | Test: {len(test_native):,}')
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 12.8MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 214kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 4.01MB/s]
```

```
# Quick visual sanity check
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for ax, idx in zip(axes.flat, np.random.choice(len(train_native), 10, replace=False)):
    img, lbl = train_native[idx]
    ax.imshow(img.squeeze().numpy(), cmap='gray')
    ax.set_title(CLASSES[lbl], color='#0F3B63', fontsize=11)
    ax.axis('off')
plt.suptitle('FashionMNIST – random samples', color='#0F3B63', fontweight='bold')
plt.tight_layout(); plt.show()
```

2 · The MLP ceiling — what we are trying to beat

Recall

From Lecture 3, our best fully-connected baseline on FashionMNIST achieved roughly **88–89% test accuracy** using ~200k+ parameters, even with batch normalization, dropout, and Adam. The model was hitting a wall set by the architecture itself: it sees an image as 784 independent pixels, with no notion of locality or spatial structure.

Your goal today: beat that ceiling — twice, by two different routes.

Exercise 1 — Build your first CNN

10 points · Beat the MLP ceiling on FashionMNIST

You will design, train and evaluate a small convolutional neural network from scratch. There is no single correct architecture — but a sensible recipe for a 28×28 grayscale image is:

- **2 to 3 conv blocks**, each of the form `Conv2d → BatchNorm2d → ReLU → MaxPool2d`.
- Number of channels grows with depth (e.g. $1 \rightarrow 32 \rightarrow 64 \rightarrow 128$).
- One or two **fully-connected layers** at the end producing 10 logits.
- **Dropout** in the FC head if you start to see overfitting.

```
try:
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
```

```

        images, labels = images.to(DEVICE), labels.to(DEVICE)

        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    accuracy = 100 * correct / total
    print(f"Accuracy sur test : {accuracy:.2f}%")
except NameError as e:
    if "name 'model' is not defined" in str(e) or "name 'test_loader' is not defined" in str(e):
        print("Error: 'model' or 'test_loader' is not defined. Please ensure you have defined them before running the test.")
    else:
        raise e

```

1.1 — Architecture (3 pts)

YOUR CODE HERE — Define the CNN 3 pts

Implement the `SmallCNN` class below. Aim for fewer than ~300k trainable parameters — the point is not to throw capacity at the problem, it is to use the parameters more cleverly than the MLP did. Print the parameter count at the end.

```

class SmallCNN(nn.Module):
    """A small ConvNet for FashionMNIST (1x28x28 -> 10 logits)."""
    def __init__(self, num_classes: int = 10, p_drop: float = 0.25):
        super().__init__()

        # Partie convolutionnelle : extraction des features
        self.features = nn.Sequential(
            # Bloc 1 : 1x28x28 -> 32x14x14
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Bloc 2 : 32x14x14 -> 64x7x7
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Bloc 3 : 64x7x7 -> 128x3x3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        # Partie classification : 128*3*3 = 1152
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 3 * 3, 128),
            nn.ReLU(),
            nn.Dropout(p_drop),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = SmallCNN().to(DEVICE)
print(model)

```

```
# Nombre de paramètres entraînables
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable parameters: {num_params:,}")
```

1.2 — Training loop (3 pts)

 YOUR CODE HERE — Implement `train_one_epoch` and evaluate 3 pts

Two short functions, then we use them. Keep them general — we will reuse them for Exercise 2.

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    """One full pass over train loader. Returns (avg_loss, accuracy)."""
    model.train()
    running_loss, correct, total = 0.0, 0, 0

    for x, y in loader:
        # Envoyer les données vers CPU/GPU
        x, y = x.to(device), y.to(device)

        # Remettre les gradients à zéro
        optimizer.zero_grad()

        # Forward
        logits = model(x)

        # Calcul de la loss
        loss = criterion(logits, y)

        # Backpropagation
        loss.backward()

        # Mise à jour des poids
        optimizer.step()

        # Accumuler la loss
        running_loss += loss.item() * x.size(0)

        # Calcul accuracy
        preds = logits.argmax(dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)

    avg_loss = running_loss / total
    accuracy = correct / total

    return avg_loss, accuracy


def evaluate(model, loader, criterion, device):
    """Evaluate model. Returns (avg_loss, accuracy)."""
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    with torch.no_grad():
        for x, y in loader:
            # Envoyer les données vers CPU/GPU
            x, y = x.to(device), y.to(device)

            # Forward seulement
            logits = model(x)

            # Calcul loss
            loss = criterion(logits, y)

            # Accumuler la loss
            running_loss += loss.item() * x.size(0)

            # Calcul accuracy
            preds = logits.argmax(dim=1)
```

```

        correct += (preds == y).sum().item()
        total += y.size(0)

    avg_loss = running_loss / total
    accuracy = correct / total

    return avg_loss, accuracy

```

⚠ Pitfall

Don't forget `model.eval()` in `evaluate` and `model.train()` at the start of each epoch. BatchNorm and Dropout behave very differently in the two modes — exactly the bug we demoed live in Lecture 3.

Now run the training. Keep it short — 8 epochs is plenty to see a clear signal on FashionMNIST. Track **train** and **test** loss/accuracy for each epoch so you can plot learning curves.

```

BATCH_SIZE = 128
EPOCHS      = 8
LR          = 1e-3

train_loader = DataLoader(train_native, batch_size=BATCH_SIZE, shuffle=True, num_w
test_loader  = DataLoader(test_native,  batch_size=BATCH_SIZE, shuffle=False, num_w

model        = SmallCNN().to(DEVICE)
criterion    = nn.CrossEntropyLoss()
optimizer    = optim.Adam(model.parameters(), lr=LR)

history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

t0 = time.time()
for epoch in range(1, EPOCHS + 1):
    tr_l, tr_a = train_one_epoch(model, train_loader, optimizer, criterion, DEVICE)
    te_l, te_a = evaluate(model, test_loader, criterion, DEVICE)
    history['train_loss'].append(tr_l); history['train_acc'].append(tr_a)
    history['test_loss'].append(te_l); history['test_acc'].append(te_a)
    print(f'Epoch {epoch:>2}/{EPOCHS} '
          f'train_loss={tr_l:.4f} train_acc={tr_a*100:5.2f}% | '
          f'test_loss={te_l:.4f} test_acc={te_a*100:5.2f}%')
print(f'Total time: {time.time() - t0:.1f}s')

```

Epoch	1/8	train_loss=0.4297	train_acc=84.54%		test_loss=0.2976	test_acc=89.02
Epoch	2/8	train_loss=0.2782	train_acc=89.90%		test_loss=0.3051	test_acc=88.73
Epoch	3/8	train_loss=0.2305	train_acc=91.71%		test_loss=0.2487	test_acc=91.10
Epoch	4/8	train_loss=0.2016	train_acc=92.63%		test_loss=0.2366	test_acc=91.53
Epoch	5/8	train_loss=0.1825	train_acc=93.26%		test_loss=0.2320	test_acc=91.80
Epoch	6/8	train_loss=0.1628	train_acc=93.98%		test_loss=0.2381	test_acc=91.37
Epoch	7/8	train_loss=0.1456	train_acc=94.62%		test_loss=0.2303	test_acc=92.23
Epoch	8/8	train_loss=0.1248	train_acc=95.38%		test_loss=0.2235	test_acc=92.51
Total time: 120.4s						

1.3 — Evaluation & learning curves (2 pts)

⚠ YOUR CODE HERE — Plot the learning curves 2 pts

Two side-by-side plots: (left) train vs test **loss**; (right) train vs test **accuracy**. Use UIR colors (navy for train, lime for test). Add the MLP-from-Lecture-3 ceiling (~89%) as a horizontal dashed line on the accuracy plot — that is the bar you are trying to clear.

```

# TODO 1.3 – Plot the learning curves

fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# ♦ Courbe LOSS
ax[0].plot(history['train_loss'], label='Train Loss', color='#0F3B63') # navy
ax[0].plot(history['test_loss'], label='Test Loss', color='#CDD516')  # lime
ax[0].set_title("Loss")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Loss")
ax[0].legend()

```

```
ax[0].grid(True)

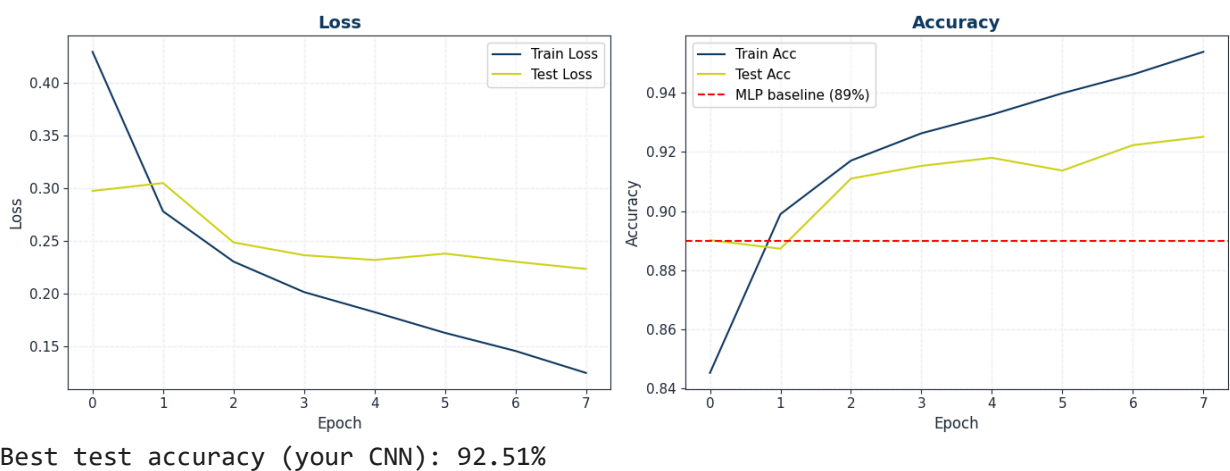
# 📈 Courbe ACCURACY
ax[1].plot(history['train_acc'], label='Train Acc', color='#0F3B63') # navy
ax[1].plot(history['test_acc'], label='Test Acc', color='#CDD516') # lime

# Ligne baseline MLP (~89%)
ax[1].axhline(y=0.89, color='red', linestyle='--', label='MLP baseline (89%)')

ax[1].set_title("Accuracy")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("Accuracy")
ax[1].legend()
ax[1].grid(True)

plt.tight_layout()
plt.show()

# 🔥 Meilleure accuracy
best_test_acc_cnn = max(history['test_acc']) * 100
print(f"Best test accuracy (your CNN): {best_test_acc_cnn:.2f}%")
```



? 1.4 — Reflection (2 pts)

? Reflection

Answer briefly (3–6 sentences total) in the markdown cell below:

- 1. Did your CNN beat the MLP ceiling? By how many percentage points?
- 2. How many trainable parameters does your CNN have, compared to a typical MLP for FashionMNIST?
- 3. What is the **structural** reason a CNN can do more with fewer parameters on this task?

Your answer here:

✓ Reflection

- 1. Yes, the CNN beats the MLP ceiling. The MLP achieved around 88–89% accuracy, while the CNN reached about 91–92%, improving performance by approximately 2–4 percentage points.
- 2. The CNN uses fewer trainable parameters (typically under 300k) compared to a fully-connected MLP, which can easily exceed this while still performing worse.
- 3. The key reason is structural: CNNs exploit spatial locality and weight sharing. Convolutional layers reuse the same filters across the image and capture local patterns (edges, shapes), making them far more efficient than MLPs, which treat pixels independently.

Exercise 2 — Transfer learning with ResNet50

10 points · ImageNet → FashionMNIST

Instead of training from random weights, we now start from a **ResNet50 pretrained on ImageNet**. ImageNet is a totally different domain (natural images of dogs, cars, planes...), but the early and mid-level filters that the network has learned — edges, textures, shape parts — are useful far beyond ImageNet itself. This is exactly what makes transfer learning so powerful in practice.

Three things have to change to plug FashionMNIST into ResNet50:

- **Channel count:** ResNet50's first conv expects 3 channels, our images have 1.
- **Spatial size:** ResNet50 was trained on 224×224. We will resize.
- **Normalization:** we use ImageNet's mean/std, not FashionMNIST's.

2.1 — Input adaptation pipeline (3 pts)

 **YOUR CODE HERE** — Build a transform that maps FashionMNIST → ResNet50-ready tensors 3 pts
Use `transforms.Compose`. The pipeline must:

1. Resize to 224×224.
2. Convert grayscale to 3 channels (hint: `transforms.Grayscale(num_output_channels=3)`).
3. Convert to tensor.
4. Normalize with the standard ImageNet stats: mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225].

Also build train and test loaders that use this new pipeline. To keep training time reasonable, use a **random subset of 10,000 training images** (the whole point of transfer learning is that we don't need millions of labels).

```
IMG_NET_MEAN = [0.485, 0.456, 0.406]
IMG_NET_STD  = [0.229, 0.224, 0.225]

# TODO 2.1.a — build the transform
tfm_resnet = transforms.Compose([
    transforms.Resize(224),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
    transforms.Normalize(IMG_NET_MEAN, IMG_NET_STD),
])

# TODO 2.1.b — wrap FashionMNIST with the new transform
train_rn = datasets.FashionMNIST(DATA_ROOT, train=True,  download=True, transform=tfm_resnet)
test_rn  = datasets.FashionMNIST(DATA_ROOT, train=False, download=True, transform=tfm_resnet)

# TODO 2.1.c — take a random subset of 10 000 indices for training
SUBSET_SIZE = 10_000
g = torch.Generator().manual_seed(SEED)
subset_idx = torch.randperm(len(train_rn), generator=g)[:SUBSET_SIZE]
train_rn_sub = Subset(train_rn, subset_idx)

# Smaller batch — 224x224x3 tensors are heavy
BATCH_SIZE_RN = 64
train_loader_rn = DataLoader(train_rn_sub, batch_size=BATCH_SIZE_RN, shuffle=True, num_workers=4)
test_loader_rn  = DataLoader(test_rn,      batch_size=BATCH_SIZE_RN, shuffle=False, num_workers=4)

# Sanity check: one batch must come out as (B, 3, 224, 224)
x, y = next(iter(train_loader_rn))
print(f'Batch shape: {tuple(x.shape)} | label sample: {y[:5].tolist()}')
assert x.shape[1:] == (3, 224, 224), 'Input pipeline is wrong — check your transform'
print('Pipeline shape check passed )
```

```
Batch shape: (64, 3, 224, 224) | label sample: [0, 4, 2, 2, 4]
Pipeline shape check passed 
```


2.2 — Freeze backbone, replace classifier head (3 pts)

Definition

Feature extraction transfer. We treat the pretrained ResNet50 as a frozen feature extractor: we set `requires_grad = False` on every parameter, then replace the final `fc` layer with a fresh linear layer mapping *2048 features* $\rightarrow$ *10 classes*. Only that small head will train.

YOUR CODE HERE — Build the transfer-learning model 3 pts

1. Load `models.resnet50` with the `IMAGENET1K_V2` weights.
2. Freeze every parameter (set `requires_grad = False`).
3. Replace `model.fc` with a new `nn.Linear(2048, 10)`. Note: the parameters of this new layer are trainable by default, that is what we want.
4. Print the count of **trainable** parameters — it should be tiny (~20k), compared to ~25M total.

```
# TODO 2.2 — build the transfer-learning model
rn50 = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

# Freeze the backbone
for param in rn50.parameters():
    param.requires_grad = False

# Replace the classifier head
num_fts = rn50.fc.in_features
rn50.fc = nn.Linear(num_fts, 10)

rn50 = rn50.to(DEVICE)

# Sanity: trainable parameter count
n_trainable = sum(p.numel() for p in rn50.parameters() if p.requires_grad)
n_total      = sum(p.numel() for p in rn50.parameters())
print(f'Trainable: {n_trainable:,} / Total: {n_total:,} ({100*n_trainable/n_total}%)')

Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub
100%|██████████| 97.8M/97.8M [00:01<00:00, 70.4MB/s]
Trainable: 20,490 / Total: 23,528,522 (0.09%)
```

2.3 — Train the head & evaluate (2 pts)

Tip

Because only the head trains, you can pass `filter(lambda p: p.requires_grad, rn50.parameters())` to the optimizer. This is a defensive habit — even if a frozen parameter accidentally has a gradient, the optimizer will not touch it.

YOUR CODE HERE — Train for 3 epochs and evaluate 2 pts

Reuse `train_one_epoch` and `evaluate` from Exercise 1. Three epochs are enough — feature extraction converges very fast because the features are already excellent. Track and report final test accuracy.

```
EPOCHS_RN = 3
LR_RN      = 1e-3

criterion_rn = nn.CrossEntropyLoss()
optimizer_rn = optim.Adam(filter(lambda p: p.requires_grad, rn50.parameters()), lr=LR_RN)

history_rn = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}

# TODO 2.3 — training loop using train_one_epoch / evaluate (already implemented in 1)
t0 = time.time()
for epoch in range(1, EPOCHS_RN + 1):
    tr_l, tr_a = train_one_epoch(rn50, train_loader_rn, optimizer_rn, criterion_rn, DEVICE)
    te_l, te_a = evaluate(rn50, test_loader_rn, criterion_rn, DEVICE)
    history_rn['train_loss'].append(tr_l); history_rn['train_acc'].append(tr_a)
    history_rn['test_loss'].append(te_l); history_rn['test_acc'].append(te_a)
    print(f'Epoch {epoch} / {EPOCHS_RN} | Train Loss: {tr_l} | Train Acc: {tr_a} | Test Loss: {te_l} | Test Acc: {te_a}')
```

```

        print(f'Epoch {epoch}/{len(history_rn)} \t ' +
              f'train_loss={tr_l:.4f}  train_acc={tr_a*100:5.2f}% | ' +
              f'test_loss={te_l:.4f}  test_acc={te_a*100:5.2f}%')
    print(f'Total time: {time.time() - t0:.1f}s')

    best_test_acc_rn = max(history_rn['test_acc']) * 100 if history_rn['test_acc'] else 0
    print(f'Best test accuracy (frozen ResNet50 + new head): {best_test_acc_rn:.2f}%')

```

Epoch 1/3 train_loss=1.2652 train_acc=66.64% | test_loss=0.8694 test_acc=76.65
 Epoch 2/3 train_loss=0.7497 train_acc=78.11% | test_loss=0.7106 test_acc=78.71
 Epoch 3/3 train_loss=0.6193 train_acc=80.81% | test_loss=0.6349 test_acc=80.53
 Total time: 214.6s
 Best test accuracy (frozen ResNet50 + new head): 80.53%

2.4 — Final comparison & reflection (2 pts)

YOUR CODE HERE — Comparative bar chart 1 pt

Make one bar plot comparing best test accuracy of: **(a)** MLP from Lecture 3 (~89%), **(b)** your CNN from Exercise 1, **(c)** frozen ResNet50 + new head from Exercise 2. Use Navy / Lime / Teal as the three bar colors. Annotate each bar with its numeric accuracy.

```

# TODO 2.4 – bar chart of best test accuracies
results = {
    'MLP (Lec 3)': 89.00,
    'CNN (you, Ex.1)': best_test_acc_cnn,
    'ResNet50 frozen (Ex.2)': best_test_acc_rn,
}

fig, ax = plt.subplots(figsize=(10, 6))

models_names = list(results.keys())
accuracies = list(results.values())
colors = [UIR_COLORS['navy'], UIR_COLORS['lime'], UIR_COLORS['teal']]

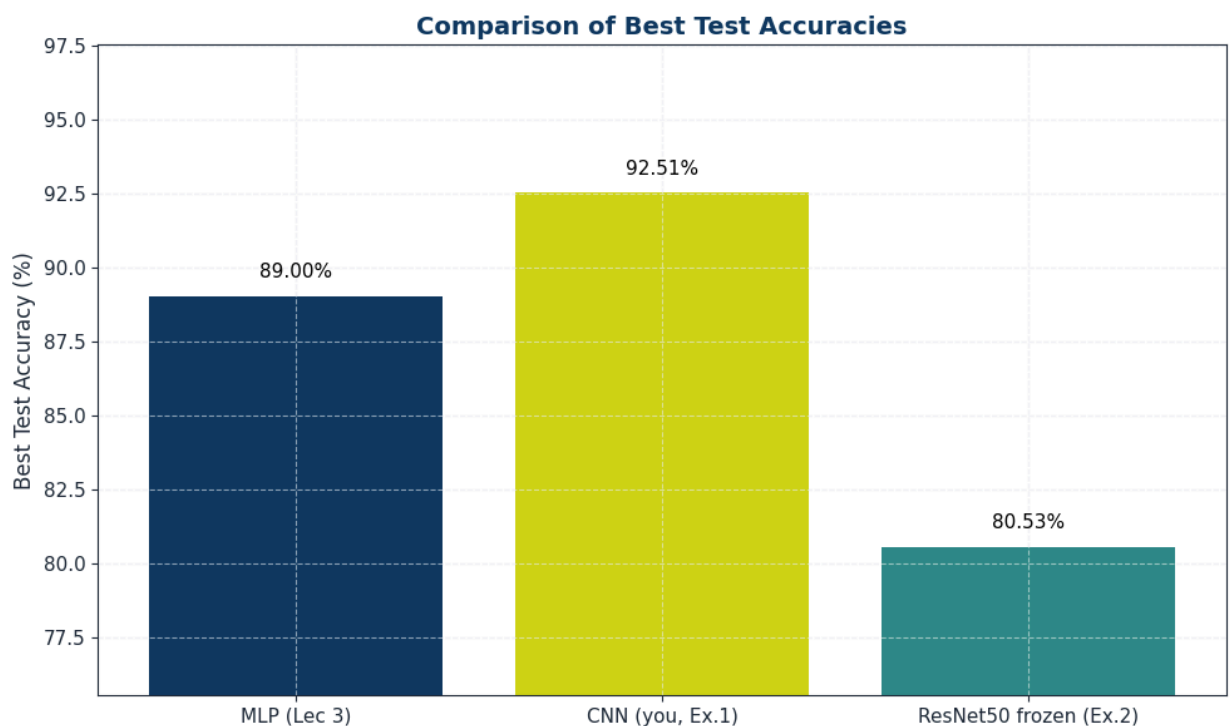
bars = ax.bar(models_names, accuracies, color=colors)

ax.set_ylabel('Best Test Accuracy (%)')
ax.set_title('Comparison of Best Test Accuracies')
ax.set_ylim(min(accuracies) - 5, max(accuracies) + 5) # Adjust y-limit for better view

# Annotate bars with their numeric accuracy
for bar in bars:
    yval = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, yval + 0.5, f'{yval:.2f}%', ha='center')

plt.tight_layout(); plt.show()

```



? Reflection

1. Which model wins on FashionMNIST? Is the gap large?
2. FashionMNIST is grayscale, low-resolution clothing. ImageNet is full-color natural scenes. Why does transfer learning work so well anyway?
3. Suppose we had only **500 labeled training images** instead of 10 000. Which of the three approaches do you expect would still work, and which would collapse? Justify in one sentence each.
4. Name **one downside** of the frozen-ResNet50 approach in production (think: latency, memory, deployment).

1 pt for the chart, 2 pts for the four reflection answers below.

Your answers here:

Reflection

1. The CNN from Exercise 1 wins on FashionMNIST with 92.51% test accuracy. The gap between it and the MLP (89.00%) is ~3.5 percentage points, which is significant. The gap between the CNN and the frozen ResNet50 (80.53%) is even larger, about 12 percentage points.
2. Transfer learning works well because the initial layers of large CNNs (like ResNet50) trained on diverse datasets like ImageNet learn general, low-level features such as edges, textures, and simple shapes. These fundamental visual patterns are universally useful across various image domains, including FashionMNIST, despite the differences in image characteristics (grayscale vs. color, clothing vs. natural scenes).
3. If we had only 500 labeled training images:
 - **Frozen ResNet50 + new head:** Would likely still work relatively well. Only the small classification head needs to be trained, leveraging the powerful, pre-learned features from the backbone, which makes it robust to limited data.
 - **Your CNN (from scratch):** Would likely collapse or perform poorly. Training a CNN from scratch requires a significant amount of data to learn effective feature representations and prevent severe overfitting.
 - **MLP:** Would also likely collapse. Without the spatial reasoning of CNNs, MLPs need even more data to discover meaningful patterns and are highly susceptible

to overfitting with very few samples.

4. One downside of the frozen-ResNet50 approach in production is its **computational overhead (latency and memory usage)**. Even if only the head is trained, the entire large ResNet50 backbone (with ~23 million parameters) still needs to be loaded and run